

数据库(4)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 13901216180/62796193

2025.10.

第3章 关系数据库标准语言SQL

3.1 SQL概述

3.2 SQL功能

3.3 SQL数据定义

3.4 SQL数据查询

3.5 SQL数据更新

3.6 空值的处理

3.7 SQL视图

3.8 SQL数据控制

3.1 SQL概述

- SQL(Structured Query Language) 结构化查询语言，是关系数据库的标准语言。
- 1974年，由Donald D. Chamberlin 和 Raymond F. Boyce 提出。
- 1979年，由IBM的San Jose研究中心在System R数据库管理系统上首次实现，称为SEQUEL（Structured English Query Language）。
- IBM 随后基于其 System R 原型开发商业产品，分别于 1979 年、1981 年和 1983 年上市了 System/38、SQL/DS 和 **DB2** 数据库管理系统。
- 特点：功能丰富、使用方便灵活、语言简洁易学，应用系统范围广，统一标准。

3.1.1 SQL标准化的历史

- 1986年10月，美国国家标准局（ANSI）的数据库委员会通过ANSI X3H2-86-61标准作为美国国家标准（草稿130页）

- 1987年ISO也批准作为国际标准：

ISO TC97 SC21 WG3 N143（简称SQL-86）

这是第一个SQL国际标准。

X3H2

Contents

Clause 1. Scope and Field of Application	
1.1 Scope	1
1.2 Field of Application	1
Clause 2. References	1
Clause 3. Overview	3
3.1 Organization	5
3.2 Notation	5
3.3 Conventions	5
3.4 Conformance	5
Clause 4. Concepts	6
4.1 Sets	9
4.2 Data Types	9
4.2.1 Character Strings	9
4.2.2 Numbers	9
4.3 Columns	10
4.4 Tables	10
4.5 Integrity Constraints	11
4.6 Schemas	11
4.7 The Database	11
4.8 Modules	12

(working draft)
Database Language
SQL
Addendum-2

May 1986

3.1.1 SQL标准化的历史

- 1989年修订版SQL-89（FIPS127-1）。（120页）
- 1992年修订版SQL-92。（622页）引入外码约束。
- 1999年修订版(ISO/IEC 9075:1999)，SQL:1999（也称为SQL3），对象关系扩展，SQL3是现在数据库商用系统的主流。（1700页）
- 2003年修订版SQL:2003。（3600页）
- 2008年修订版SQL:2008 (3777页，主要是增加了SQL外围的内容，像框架、调用接口等)。
- 2011年修订版(ISO/IEC 9075:2011)，SQL:2011，时间数据处理。
- 2016年修订版(ISO/IEC 9075:2016)，SQL:2016。

3.1.1 SQL标准化的历史

- 2019年修订版(ISO/IEC 9075:2019), SQL:2019。增加了第15 部分: ISO/IEC 9075-15:2019 多维数组 (SQL/MDA)。多维数组 (Multi-Dimensional Arrays) 是各种科学和工程数据的核心基础结构。 SQL/MDA允许存储、访问和处理规模的多维数组, 例如 N 通道的卫星图像。这意味着 SQL 现在可以解码图像, 并且通过像素坐标直接访问和处理图像区域。
- 2021年修订版: ISO/IEC 9075-8:2021 Information technology — Guidance for the use of database language SQL — Part 8: Multidimensional arrays 多维数组 已经重新编号为第8部分。
- ISO —— 国际标准化组织
- IEC —— 国际电工委员会
- CD —— 委员会草案
- COR——勘误表

以下是2021 SQL 标准的组成部分，其中有一些编号已被弃用：

- ISO/IEC 9075-1 信息技术 – 数据库语言 – SQL – 第 1 部分：框架（SQL/框架）
- ISO/IEC 9075-2 信息技术 – 数据库语言 – SQL – 第 2 部分：基本原则（SQL/基本原则）
- ISO/IEC 9075-3 信息技术 – 数据库语言 – SQL – 第 3 部分：调用级接口（SQL/CLI）
- ISO/IEC 9075-4 信息技术 – 数据库语言 – SQL – 第 4 部分：持久存储模块（SQL/PSM）
- ISO/IEC 9075-8 信息技术 – 数据库语言 – SQL – 第 15 部分：多维数组（SQL/MDA）
- ISO/IEC 9075-9 信息技术 – 数据库语言 – SQL – 第 9 部分：外部数据管理（SQL/MED）
- ISO/IEC 9075-10 信息技术 – 数据库语言 – SQL – 第10 部分：对象语言绑定（SQL/OLB）
- ISO/IEC 9075-11 信息技术 – 数据库语言 – SQL – 第 11 部分：信息与定义概要（SQL/Schemata）
- ISO/IEC 9075-13 信息技术 – 数据库语言 – SQL – 第 13 部分：使用 Java 编程语言的 SQL 程序与类型（SQL/JRT）
- ISO/IEC 9075-14 信息技术 – 数据库语言 – SQL – 第 14 部分：XML 相关规范（SQL/XML）

■2023年6月发布了最新修订版：
ISO/IEC 9075:2023 Information
technology -Database languages
SQL

■SQL:2023 标准中有177 项强制功能。最新标准中的型特性，主要分为以下三大类别：

(1) 已有 SQL 语言的增强

(2) JSON 相关的新特性

(3) 新增加属性图查询语言

该版本新增功能都属于可选特性。

■左面是：ISO/IEC 9075-9:2023 第9部分：Management of External Data (SQL/MED)的封面，后面2页ppt是该文本的内容。

INTERNATIONAL STANDARD

ISO/IEC 9075-9

Fifth edition
2023-06

Information technology — Database languages SQL —

Part 9: Management of External Data (SQL/ MED)

*Technologies de l'information — Langages de base de données
SQL —*

Partie 9: Gestion des données externes (SQL/MED)



Reference number
ISO/IEC 9075-9:2023(E)

© ISO/IEC 2023

The organization of this document is as follows:

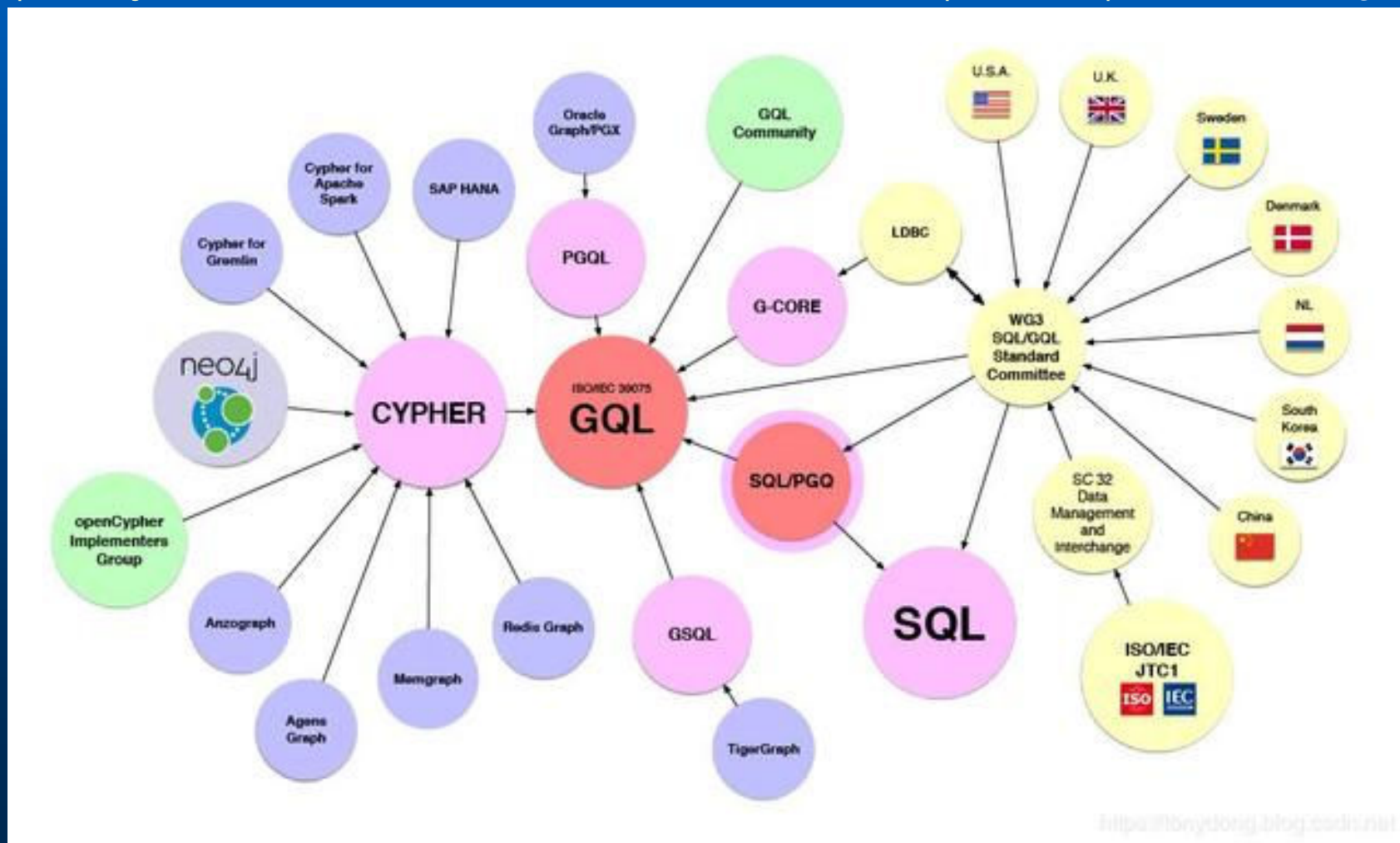
- 1) Clause 1, "Scope", specifies the scope of this document.
- 2) Clause 2, "Normative references", identifies additional standards that, through reference in this document, constitute provisions of this document.
- 3) Clause 3, "Terms and definitions", defines the terms and definitions used in this document.
- 4) Clause 4, "Concepts", presents concepts related to this document.
- 5) Clause 5, "Lexical elements", defines the lexical elements of the language specified in this document.
- 6) Clause 6, "Scalar expressions", defines the elements of the language that produce scalar values.
- 7) Clause 7, "Query expressions", defines the elements of the language that produce rows and tables of data.
- 8) Clause 8, "Additional common rules", specifies the rules for assignments that retrieve data from or store data into SQL-data, and formation rules for set operations.
- 9) Clause 9, "Additional common elements", defines additional common elements used in the definition of foreign tables, foreign servers, and foreign-data wrappers.
- 10) Clause 10, "Schema definition and manipulation", defines facilities related to foreign tables and datalink type support for creating and managing a schema.
- 11) Clause 11, "Access control", defines facilities for controlling access to SQL-data.
- 12) Clause 12, "SQL-client modules", defines SQL-client modules and externally-invoked procedures.
- 13) Clause 13, "Additional data manipulation rules", defines additional rules for data manipulation.
- 14) Clause 14, "Session management", defines the SQL-session management statements.
- 15) Clause 15, "Dynamic SQL", defines the dynamic SQL statements.
- 16) Clause 16, "Embedded SQL", defines the embedded SQL statements.
- 17) Clause 17, "Diagnostics management", defines the diagnostics management facilities.
- 18) Clause 18, "Call-Level Interface specifications", defines facilities for using SQL through a Call-Level Interface.
- 19) Clause 19, "SQL/CLI routines", defines each of the routines that comprise the Call-Level Interface.
- 20) Clause 20, "URLs", specifies the format of URLs used in this document.
- 21) Clause 21, "Catalog manipulation", defines facilities for creating, altering, and dropping foreign servers and foreign-data wrappers.
- 22) Clause 22, "SQL/MED common specifications", specifies common facilities used by SQL/MED.
- 23) Clause 23, "Foreign-data wrapper interface routines", specifies the interaction between an SQL-server and a foreign-data wrapper.
- 24) Clause 24, "Information Schema", defines viewed tables that contain schema information.
- 25) Clause 25, "Definition Schema", defines base tables on which the viewed tables containing schema information depend.

- 26) [Clause 26, “Status codes”](#), defines values that identify the status of the execution of SQL-statements and the mechanisms by which those values are returned.
- 27) [Clause 27, “Conformance”](#), specifies the way in which conformance to this document may be claimed.
- 28) [Annex A, “SQL conformance summary”](#), is an informative Annex. It summarizes the conformance requirements of the SQL language.
- 29) [Annex B, “Implementation-defined elements”](#), is an informative Annex. It lists those features for which the body of document states that the syntax, the meaning, the returned results, the effect on SQL-data and/or schemas, or other aspect is partly or wholly implementation-defined.
- 30) [Annex C, “Implementation-dependent elements”](#), is an informative Annex. It lists those features for which the body of this document states that the syntax, the meaning, the returned results, the effect on SQL-data and/or schemas, or other aspect is partly or wholly implementation-dependent.
- 31) [Annex D, “SQL optional feature taxonomy”](#), is an informative Annex. It identifies the optional features of the SQL language specified in this document by an identifier and a short descriptive name. This taxonomy is used to specify conformance.
- 32) [Annex E, “Deprecated features”](#), is an informative Annex. It lists features that the responsible Technical Committee intends not to include in a future edition of this document.
- 33) [Annex F, “Incompatibilities with ISO/IEC 9075:2016”](#), is an informative Annex. It lists incompatibilities with the previous edition of this document.
- 34) [Annex G, “Defect Reports not addressed in this edition of this document”](#), is an informative Annex. It describes the Defect Reports that were known at the time of publication of this document. Each of these problems is a problem carried forward from the previous edition of this document. No new problems have been created in the drafting of this edition of this document.
- 35) [Annex H, “Example header files”](#), is an informative Annex. It provides examples of typical definition files for application programs using the SQL Call-Level Interface.
- 36) [Annex I, “SQL/MED model”](#), is an informative Annex. It uses annotated diagrams to illustrate the more important concepts of the model of SQL/MED, including the relationships between the SQL-server, foreign-data wrappers, and foreign servers.

In the text of this document, Clauses and Annexes begin new odd-numbered pages, and in [Clause 5, “Lexical elements”](#), through [Clause 27, “Conformance”](#), Subclauses begin new pages. Any resulting blank space is not significant.

属性图查询语言是 SQL 2023标准中增加的一个全新部分：**Property Graph Queries (SQL/PGQ)**。这个新功能支持使用图数据库的方式查询表中的数据。此前，Neo4j 于 2019 年 9 月 17 宣布图形查询语言（**GQL**）成为了继 SQL 之后另一种新的 ISO 标准数据库查询语言。新的第 16 部分（**SQL/PGQ**）主要是为了在 SQL 中直接提供一些 **GQL** 功能。

参见：<https://baijiahao.baidu.com/s?id=1660253880326375319&wfr=spider&for=pc> 春眠不觉晓，SQL 知多少？



到目前（2025年10月）为止，SQL:2023标准已发布两年，其新增的图查询功能正逐步在PostgreSQL16等主流数据库中实现。

中国作为ISO成员国参与标准制定，国内数据库产品如GaussDB、OceanBase已实现SQL-92及以上标准兼容。

SQL:2023新标准的图形查询能力与GQL国际标准(ISO/IEC 39075:2024)形成技术协同。

3.1.2 SQL主要特点

1. 一体化

- 集DDL，DML，DCL于一体，可以完成从建立数据库，定义、删除和修改关系模式，定义、删除和修改视图，**插入、删除**数据，**查询**数据，**更新**数据，数据库重构和维护，安全性控制（权限管理），完整性控制，事务控制（并发控制）等全部操作。使得数据库系统具有良好的可维护性和可扩展性。
- 单一的结构----**关系**，带来了数据操作符的统一，查询实体和联系都是查表而已。查找、插入、删除、更新等每一种操作都只需一种操作符。

3.1.2 SQL主要特点

2. 高度非过程化

SQL语言进行数据操作，只需提出“做什么”，无须指明“怎么做”，无须了解存取路径。存取路径的选择以及SQL的操作规程全部由系统自动完成。这不但大大减轻了用户负担，而且有利于提高数据独立性。

附：SQL程序中可以像C/C++源程序添加注释，SQL程序中使用 `--` 开始的行或者 `/*` 和 `*/` 之间的内容表示注释。注释不会被执行，用于帮助我们理解代码，增加可读性。

3.1.2 SQL主要特点

3. 面向集合的操作方式

SQL语言采用集合操作方式，不仅操作对象、查找结果可以是元组的集合，而且一次插入、删除、更新操作的对象也可以是元组的集合。

因此操作不再是过程化的一次一记录，而是非过程化的一次一集合。

3.1.2 SQL主要特点

4. 两种使用方式，统一的语法结构：自主型和宿主型

自主型：又称为交互式，联机交互式，解释方式执行SQL语句。

宿主型：又称为嵌入式，通过编译后执行，将SQL语句嵌入到某种高级语言(C、C++、Java，Python等)中，这需要数据库厂家修改高级语言编译器，能够识别SQL语句。(说明：由于ODBC和JDBC的出现，现在在Win平台上基本不再用嵌入式，而是以字符串形式将SQL语句通过ODBC或JDBC直接发送到数据库服务器上执行。好处是不再需要数据库厂家修改高级语言编译器。)

3.1.2 SQL主要特点

5. 语言简洁，易学易用

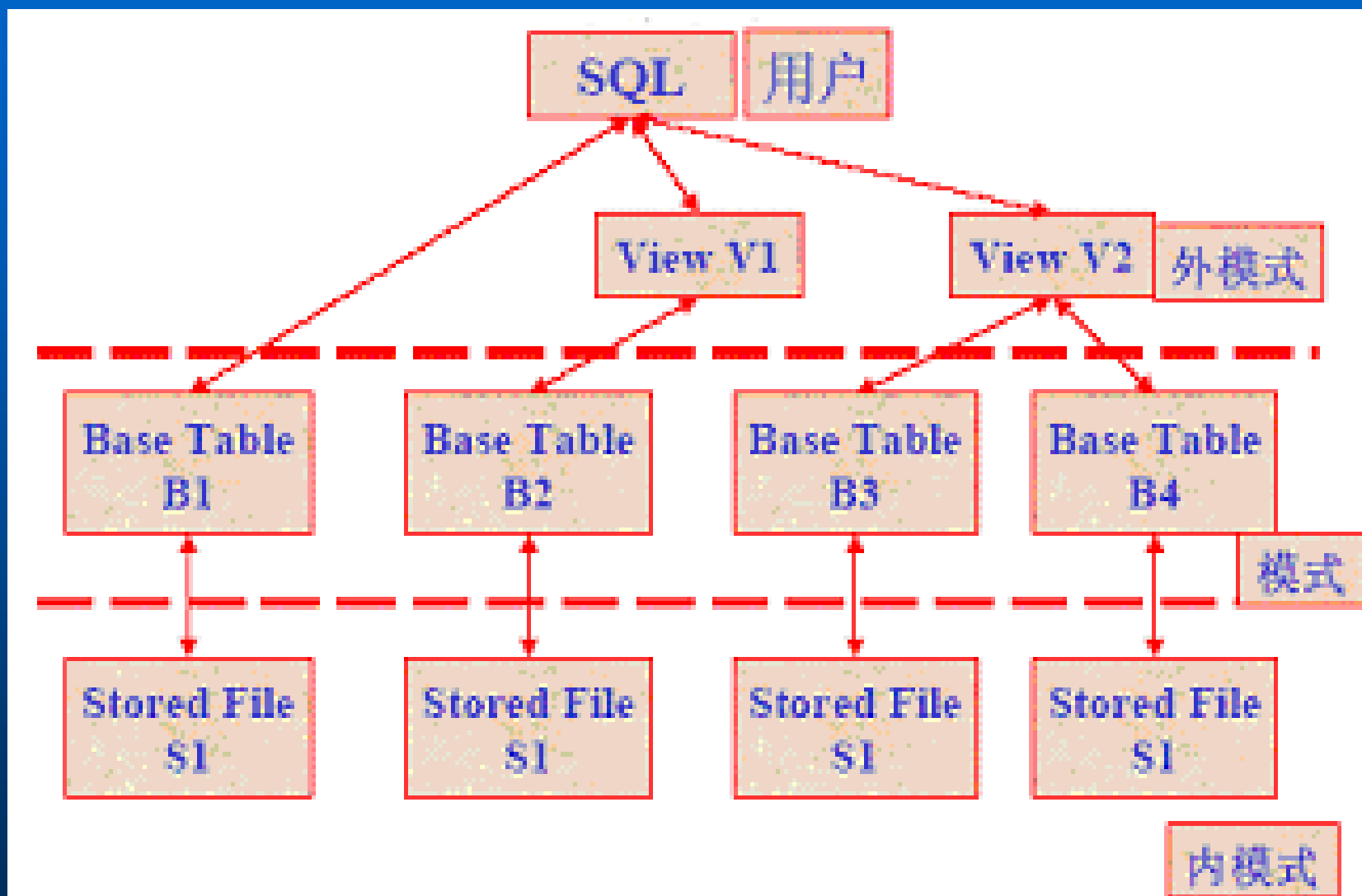
SQL功能强大，设计非常简洁巧妙，核心功能仅用9个动词。而且SQL很接近英语口语，因此易于学习、理解和使用。

SQL功能	操作符（动词）
数据查询	SELECT
数据定义	CREATE, DROP, ALTER
数据操纵	INSERT, UPDATE, DELETE
数据控制	GRANT, REVOKE

3.1.3 SQL体系结构和基本概念

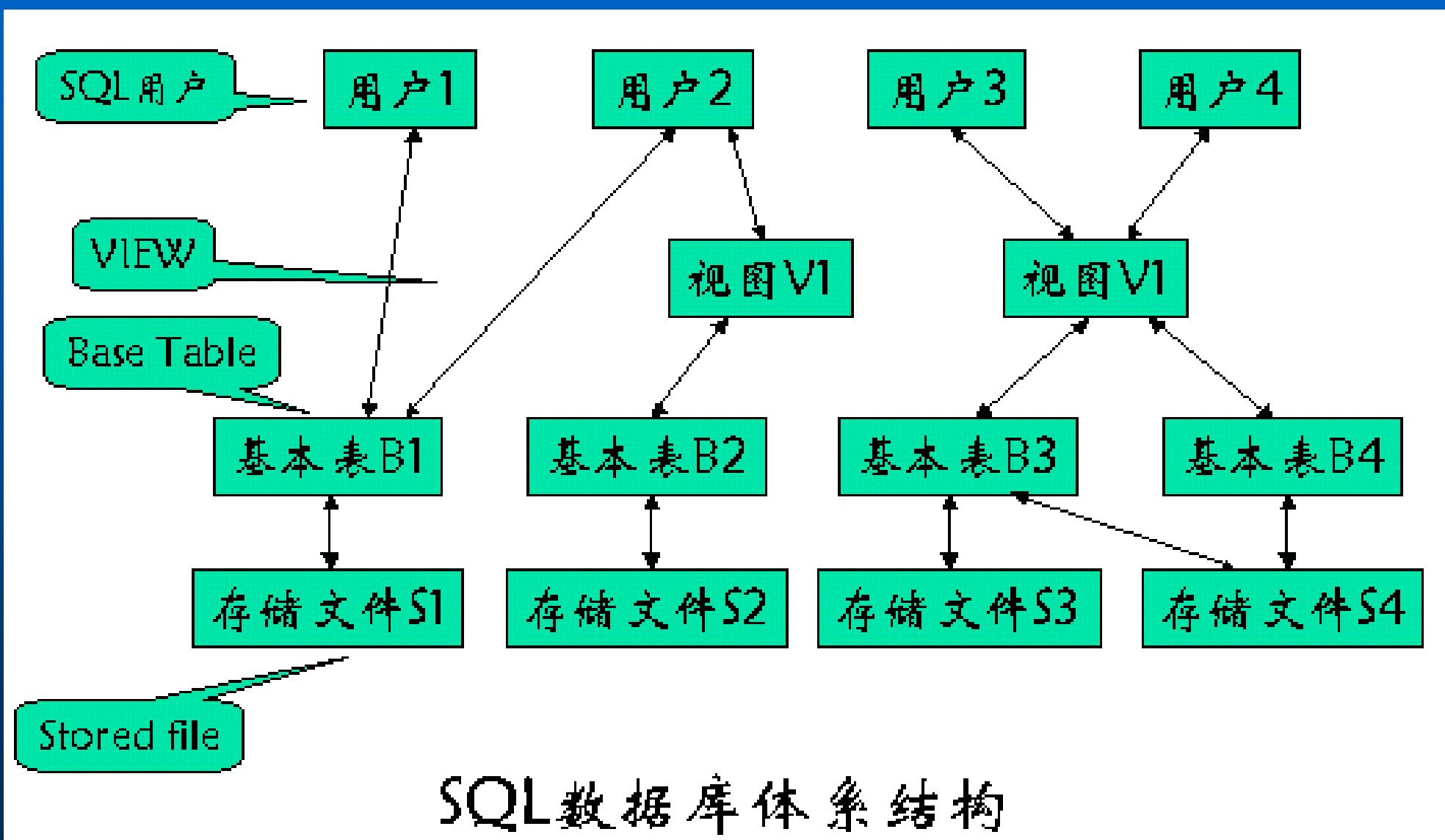
- SQL支持数据库三级模式结构：
内模式（存储文件、索引文件）
模式（基本表）
外模式（基本表和视图）
- **基本表** (*Base Table*) 本身独立存在的表，每个基本表对应一个存储文件和若干个索引文件。在关系数据库中，每个关系对应一个基本表。
- **视图** (*View*) 是从一个或多个基本表或其他视图导出的表，是**虚表**，本身不独立存在，仅在数据字典（DD）中存储视图的定义。

3.1.3 SQL体系结构和基本概念



SQL语言支持的关系数据库的三级模式结构

3.1.3 SQL体系结构和基本概念



3.2 SQL功能

- 数据定义（DDL）

- 定义、删除模式（Schema）
- 定义、删除、修改关系模式（Table 基本表）
- 定义、删除视图（View）
- 定义、删除索引（Index）

- 数据查询

- 数据操纵（DML）

- 数据插入、删除、修改

- 数据控制（DCL）

- 用户访问权限的授予、收回

3.3 SQL数据定义

3.3.1 模式的定义与删除

1. 定义模式 (CREATE SCHEMA)

语句格式:

CREATE SCHEMA [<模式名>] AUTHORIZATION <用户名>;

- 如果没指定<模式名>, 那么系统隐含用<用户名>生成同名的模式。
- 想用此命令创建模式, 必须拥有DBA的权限, 或者获得了DBA授予的CREATE SCHEMA 的权限。

例如: CREATE SCHEMA S-T AUTHORIZATION WANG;
为用户WANG定义了一个学生-课程模式S-T。

CREATE SCHEMA AUTHORIZATION WANG;
为用户WANG定义了一个模式WANG。

- 定义模式实际上定义了一个类似C++的命名空间, 在这个空间里可以进一步定义基本表、视图、索引等。

3.3 SQL数据定义

3.3.1 模式的定义与删除

- 方法是，可以在定义模式的同时生成基本表等，或者在生成基本表、视图和索引时指定他们所隶属的模式。

例如： `CREATE SCHEMA TEST AUTHORIZATION ZHANG
CREATE TABLE STU(SNO INT,
NAME CHAR(20),
.....);`

为用户ZHANG定义了一个模式TEST的同时，在TEST模式中定义了一个基本表STU。

或者： `CREATE SCHEMA TEST AUTHORIZATION ZHANG;
CREATE TABLE TEST.STU(SNO INT,
NAME CHAR(20),
.....);`

先为用户ZHANG定义一个模式TEST，再在TEST模式中定义一个基本表STU。

3.3 SQL数据定义

3.3.1 模式的定义与删除

2. 删除模式(DROP SCHEMA)

语句格式:

`DROP SCHEMA <模式名> CASCADE | RESTRICT;`

- 其中 CASCADE 和 RESTRICT 这两者必须选一个。
- 选择 CASCADE（级联）表示在删除模式的同时把该模式中的所有数据库对象（基本表、视图等）全部删除。
- 选择 RESTRICT（限制）表示如果该模式中已经定义了数据库对象（基本表、视图等），则拒绝该删除语句的执行。也就是说，只有先把该模式中的数据库对象（基本表、视图等）删空后，才能执行此DROP SCHEMA 语句删除模式。
- 例如： `DROP SCHEMA TEST CASCADE;`
删除了模式TEST，该模式中已经定义的表STU也一起被删除了。

3.3.2 基本表的定义、删除与修改

1. 定义基本表 (CREATE TABLE)

语句格式:

```
CREATE TABLE <表名> (<列名1><数据类型>[列级完整性约束条件]  
                    [, <列名2><数据类型>[列级完整性约束条件]]  
                    ...  
                    [, <表级完整性约束条件>]);
```

- 列级完整性约束条件:

NULL(空)、NOT NULL(非空)、UNIQUE(取值唯一)、
PRIMARY KEY (主码)、**CHECK**等。

例如: NOT NULL UNIQUE 表示取值唯一, 不能取空值。

- 表级完整性约束条件:

FOREIGN KEY 列名1 REFERENCES 表名(列名2) 关联外码
PRIMARY KEY (列名1,列名2[,...,列名n]) 定义多属性主码

数据类型	含义
CHAR(n), CHARACTER(n)	长度为n的定长字符串
VARCHAR(n), CHARACTERVARYING(n)	最大长度为n的变长字符串
CLOB	字符串大对象
BLOB	二进制大对象
INT, INTEGER	长整型（4字节）
SMALLINT	短整型（2字节）
BIGINT	大整型（8字节）
DECIMAL(p,d), DEC(p,d), NUMBER(p,d)	定点数，由p位数字（不包括符号位、小数点）组成，小数点后面有d位数字
REAL	取决于机器精度的单精度浮点数
DOUBLE PRECISION	取决于机器精度的双精度浮点数
FLOAT(n)	可选精度的浮点数，精度至少为n位数字
BOOLEAN	逻辑布尔量
DATE	日期，包括年月日，格式为 YYYY-MM-DD
TIME	时间，包括时分秒，格式为 HH:MM:SS
TIMESTAMP	时间戳类型
INTERVAL	时间间隔类型

3.3.2 基本表的定义、删除与修改

- 生成学生关系S(SNO, Sname, Sex, Age, Dept)

```
CREATE TABLE S (  
    SNO CHAR(10) NOT NULL UNIQUE,  
    SNAME VARCHAR(14) NULL,  
    SEX CHAR(1) NULL,  
    AGE SMALLINT NULL,  
    DEPT CHAR(15) NULL);
```

- **注意：** SQL语言不区分字母的大小写

- 考虑关系数据库三类完整性，生成学生-课程关系数据库：

学生：S(SNO, Sname, Sex, Age, Dept)

课程：C(CNO, Cname, Cpno)

选修：SC(SNO, CNO, Grade)

CREATE TABLE S

(SNO CHAR(10) PRIMARY KEY,

SNAME VARCHAR(14) NULL,

SEX CHAR(1) NOT NULL,

AGE SMALLINT NOT NULL,

DEPT CHAR(15) NULL

);

```
CREATE TABLE C  
( CNO CHAR(10) PRIMARY KEY,  
  CNAME VARCHAR(40) NOT NULL,  
  CPNO CHAR(10) NULL,  
  FOREIGN KEY CPNO REFERENCES C(CNO)  
);
```

```
CREATE TABLE SC  
( SNO CHAR(10),  
  CNO CHAR(10),  
  GRADE SMALLINT NULL,  
  PRIMARY KEY (SNO, CNO),  
  FOREIGN KEY SNO REFERENCES S(SNO),  
  FOREIGN KEY CNO REFERENCES C(CNO)  
);
```

3.3.2 基本表的定义、删除与修改

2. 模式与基本表

- 如果已经定义了模式，每一个基本表都隶属于某一个模式，而一个模式会包含多个基本表。

- 定义基本表时一般可以有三种方法定义它所属的模式。

- 方式一，在表名中加上所属的模式名。

```
CREATE TABLE TEST.STU(SNO INT, NAME CHAR(20), ..... );
```

- 方式二，在创建模式时同时创建表。

```
CREATE SCHEMA TEST AUTHORIZATION ZHANG (注意这里无分号)
```

```
CREATE TABLE STU(SNO INT, NAME CHAR(20), ..... );
```

- 方式三，设置所属的模式,这样在创建表时就不必再写模式名。

- ✓ 当用户创建基本表时，若没有指定所属模式，系统会根据搜索路径（search path）来确定该对象所属的模式。

- ✓ 可以用 SHOW search_path; 显示当前的搜索路径，如果想改变搜索路径，则可以用SET命令设置新的搜索路径，随后生成表：

```
SET search_path TO TEST, PUBLIC;
```

```
CREATE TABLE STU(SNO INT, NAME CHAR(20), ..... );
```

3.3.2 基本表的定义、删除与修改

3. 修改基本表

ALTER TABLE <表名>

[ADD [COLUMN] <新列名> <数据类型>[完整性约束]]

[ADD <表级完整性约束>]

[DROP [COLUMN] <列名> [CASCADE | RESTRICT]]

[DROP CONSTRAINT <完整性约束名> [RESTRICT | CASCADE]]

[RENAME COLUMN <列名> TO <新列名>]

[ALTER COLUMN <列名> TYPE <数据类型> [完整性约束]];

- ADD用于增加新列、新的列级完整性约束条件和新的表级完整性约束条件。
- DROP [COLUMN] 用于删除表中的列。
- 如果指定了CASCADE，则自动删除引用了该列的其他对象，如视图。
- 如果指定了RESTRICT，如果有其他对象引用该列，则系统拒绝删除该列。
- DROP CONSTRAINT 用于删除指定的完整性约束条件。
- RENAME COLUMN 子句用于修改列名。
- ALTER COLUMN 用于修改原有的列定义，可修改列名的类型和完整性约束条件。

3.3.2 基本表的定义、删除与修改

3. 修改基本表

例如:

```
ALTER TABLE S ADD Email varchar(30);
```

此时表中所有Email都为NULL

```
ALTER TABLE S DROP unique(SNAME);
```

去掉S表中SNAME属性的唯一性约束

```
ALTER TABLE S ALTER COLUMN Age INT;
```

将S表中Age属性的数据类型改为INT

3.3.2 基本表的定义、删除与修改

3. 删除基本表

DROP TABLE <表名> [RESTRICT | CASCADE];

- 如果指定了RESTRICT，如果有其他表的约束引用该表，比如CHECK、FOREIGN KEY等约束，或该表上有视图、触发器（trigger）、存储过程或函数，则系统会拒绝删除该表。
- 如果指定了CASCADE，则无条件删除该基本表，并且会自动删除引用了该表的其他对象，如视图等，但不同系统有不同约定。
- 由于基本表一旦被删除，不但表中数据和定义被删除，而且在此表上建立的索引、触发器等会被一并删除。有的系统会把在此表上建立的视图也删除掉。甚至引用该基本表的其他基本表也可能被删除，因此删除基本表事关重大，牵一发而动全身，不到万不得已，不要轻易删除基本表。

例如： **DROP TABLE S CASCADE;**

则无条件删除基本表S，并且会自动删除引用了该表的其他对象，如视图等。

3.3.3 索引的建立、删除与更新

1. 建立索引

目的：加快查询速度，可以为一个表建立多个索引。

索引包括：顺序索引、B+树索引，哈希索引、位图索引等。

语句格式：

```
CREATE [UNIQUE] [CLUSTER] INDEX <索引名>  
ON <表名> (<列名1>[次序] [,<列名2>[次序]].....);
```

UNIQUE：每一个索引值只对应唯一的记录

CLUSTER：聚簇索引

次序：ASC 或 DESC 升序或降序，缺省默认为 ASC

例如：

```
create unique index ssno on S(SNO);
```

```
create unique index sno_cno on SC(SNO ASC, CNO DESC);
```

```
create unique index ccno on C(CNO);
```

索引建立后由系统维护和使用，不需用户干预。

3.3.3 索引的建立、删除与更新

2. 更新索引

语句格式:

```
ALTER INDEX <旧索引名> RENAME TO <新索引名>;
```

例如:

```
ALTER INDEX ssno RENAME TO sssno;
```

3. 删除索引

语句格式:

```
DROP INDEX <索引名> [on <关系名>];
```

例如:

```
DROP INDEX ssno [on S];
```

```
DROP INDEX sno_cno [on SC];
```

3.2.4 数据字典

- 数据字典（Data Dictionary - DD）是关系数据库管理系统内部的一组系统表，它记录了数据库中所有的定义信息，包括关系模式定义、视图定义、索引定义、完整性约束定义、各类用户对数据库的操作权限、统计信息等。
- 关系数据库管理系统在执行SQL的数据定义语句时，实际上就是把这些定义信息存入系统的数据字典以更新其中的相应信息。
- 在进行查询处理和查询优化时，关系数据库管理系统要根据数据字典中的信息执行处理算法和优化算法，因此数据字典是关系数据库管理系统运行的重要依据。

3.4 SQL数据查询

- 查询即检索操作，是对已经存在的基本表及视图进行数据检索，不改变数据本身。
- 数据查询是数据库的核心操作。
- SQL语言提供了SELECT语句进行数据查询。
- SELECT语句具有灵活的使用方式和丰富强大的功能。

SELECT-FROM-WHERE句型

■ 基本结构

Select <列名>[,<列名>] （指出要检索的数据项）

From <表名>[,<表名>] （给出要操作的关系表名）

Where <检索条件> （给出查询结果应满足的条件）

Select $A_1, A_2, \dots, A_n$

From $r_1, r_2, \dots, r_m$

Where F

$\Leftrightarrow \prod_{A_1, A_2, \dots, A_n} (\sigma_F(r_1 \bowtie r_2 \bowtie \dots \bowtie r_m))$

SELECT的一般格式

```
SELECT [ALL|DISTINCT] 目标列表达式[[AS] 别名][,...]  
FROM <基本表名或视图> [[AS] 别名][,...]  
[WHERE 条件表达式]  
[GROUP BY 列名 [HAVING 内部函数表达式] ]  
[ORDER BY 列名 [ASC | DESC] ];  
[LIMIT <行数1> [OFFSET <行数2>]];
```

- 目标列表达式：可以为 列名，* ，算术表达式，聚集函数
“*”：表示“所有的属性”
- 内部函数表达式：GROUP BY分组的附加条件
- 结果的排序形式：ASC 升序， DESC降序
- LIMIT 限制SELECT语句查询结果的行数，OFFSET是在计算<行数1>之前首先忽略的行数。LIMIT是与GROUP BY 和 ORDER BY 并列的子句。
- WHERE后的 条件表达式：查询必须要满足的条件

WHERE 条件表达式

常用的查询条件

查询条件	谓词
比较大小	=, >, <, >=, <=, !=, <>, !>, !<
确定范围	BETWEEN AND, NOT BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
是否空值	IS NULL, IS NOT NULL
多重条件	AND, OR, NOT

3.4.1 简单查询

- 数据库： S(SNO, Sname, Sex, Age, Dept)
 C(CNO, Cname, Cpno)
 SC(SNO, CNO, Grade)

例1. 查全体学生的学号、姓名。

```
SELECT SNO, Sname  
FROM S;
```

例2. 查选修了课程的学生学号。

```
SELECT distinct SNO    (单一化, 去重复)  
FROM SC;
```

例3. 查全体学生的全部个人信息。

```
SELECT *  
FROM S;
```

3.4.1 简单查询

例4. 查全体学生的学号和出生年份。

```
SELECT SNO, 2025—Age  
FROM S;
```

给目标列起别名:

```
SELECT SNO, 2025—Age [AS] Birthday  
FROM S;
```

例5. 查选修了C1课程的学生学号和成绩，结果按照分数降序列出。

```
SELECT SNO, Grade  
FROM SC  
WHERE CNO='C1'  
ORDER BY Grade DESC;
```

3.4.1 简单查询

例6. 查年龄20~22之间的学生的学号和年龄。

```
SELECT SNO, Age
```

```
FROM S
```

```
WHERE Age BETWEEN 20 and 22;
```

表示Age在区间[20,22]中,

等价于 $\text{Age} \geq 20$ and $\text{Age} \leq 22$

因此上面的查询也可以写为:

```
SELECT SNO, Age
```

```
FROM S
```

```
WHERE Age  $\geq$  20 and Age  $\leq$  22;
```

3.4.1 简单查询

而

```
SELECT SNO, Age
```

```
FROM S
```

```
WHERE Age NOT BETWEEN 20 and 22;
```

表示Age不在区间[20,22]中,

等价于 $\text{Age} < 20 \text{ OR } \text{Age} > 22$

因此上面的查询也可以写为:

```
SELECT SNO, Age
```

```
FROM S
```

```
WHERE Age < 20 OR Age > 22;
```

3.4.1 简单查询

例7. 查信息学院的学生的信息。

```
SELECT *
```

```
FROM S
```

```
WHERE Dept in ('EE', 'CS', 'AU', 'ME', 'SS');
```

表示Dept在集合('EE', 'CS', 'AU', 'ME', 'SS')中,

等价于 Dept='EE' OR Dept='CS' OR Dept='AU'
OR Dept='ME' OR Dept='SS'

因此上面的查询也可以写为:

```
SELECT *
```

```
FROM S
```

```
WHERE Dept='EE' OR Dept='CS' OR Dept='AU'  
OR Dept='ME' OR Dept='SS' ;
```

3.4.1 简单查询

而: SELECT *
FROM S

WHERE Dept **not in** ('EE', 'CS', 'AU', 'ME', 'SS') ;

表示Dept不在集合('EE', 'CS', 'AU', 'ME', 'SS')中,

等价于 Dept!='EE' AND Dept!='CS' AND
Dept!='AU' AND Dept!='ME' AND
Dept!='SS'

因此上面的查询也可以写为:

SELECT *
FROM S

WHERE Dept!='EE' AND Dept!='CS' AND
Dept!='AU' AND Dept!='ME' AND
Dept!='SS' ;

3.4.1 简单查询

例8. 查姓名以字母D开头的学生信息。

```
SELECT *  
FROM S  
WHERE Sname LIKE 'D%';
```

谓词LIKE的一般形式： 列名 LIKE 字符串常数

谓词NOT LIKE的一般形式： 列名 NOT LIKE 字符串常数

列名：必须是字符串类型或变长字符串类型

字符串常数：(1) 字符 '_' 表示单个字符

(2) 字符 '%' 表示任意长(可以0个)字符串

(3) 其它字符只代表自己

```
SELECT *  
FROM S  
WHERE Sname LIKE '___三%';
```

3.4.1 简单查询

转义：若原来的字符串中包含'%'、'_'，需要用：

ESCAPE '<换码符>'

```
SELECT *
```

```
FROM C
```

```
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';
```

找课名为 DB_Design课的信息。

```
SELECT *
```

```
FROM C
```

```
WHERE Cname LIKE '\%\\%\\_%' ESCAPE '\';
```

找课名为符号%%开头、符号_结尾的课的信息。

3.4.1 简单查询

例9. 查缺考试成绩的学生学号和课号。

```
SELECT SNO,CNO  
FROM SC  
WHERE Grade IS NULL;
```

注意：这里IS不能用=代替，完全不等价！

而

```
SELECT SNO,CNO  
FROM SC  
WHERE Grade IS NOT NULL;
```

是查不缺考试成绩的学生学号和课号。

3.4.1 简单查询 集函数

SQL常用的集函数:

COUNT(*)

统计元组个数

COUNT([DISTINCT|ALL]<列名>) 对一列中的值统计个数

SUM([DISTINCT|ALL]<列名>) 求列值的和 (数值型)

AVG([DISTINCT|ALL]<列名>) 求列的平均值 (数值型)

MAX([DISTINCT|ALL]<列名>) 求列的最大值 (数值型)

MIN([DISTINCT|ALL]<列名>) 求列的最小值 (数值型)

例10. 查电子系学生的总人数。

```
SELECT COUNT(*)
```

```
FROM S
```

```
WHERE Dept='EE';
```

3.4.1 简单查询 集函数

例11. 查选修了课程的学生的人数。

```
SELECT COUNT(DISTINCT SNO)
FROM SC;
```

注意：这里必须先用DISTINCT去重复，防止重复统计。

例12. 求电子系学生的平均年龄。

```
SELECT AVG(Age)
FROM S
WHERE Dept='EE';
```

例13. 求选修C1课的最高分和最低分。

```
SELECT MAX(Grade), MIN(Grade)
FROM SC
WHERE CNO='C1';
```

注意：聚集函数不能出现在WHERE中作为条件表达式，而只能用于SELECT子句和GROUP BY的Having子句中。

3.4.1 简单查询 GROUP BY 子句

查询结果的分组（GROUP BY）

例14. 查每门课的选修人数。

```
SELECT CNO, COUNT(SNO)
FROM SC
GROUP BY CNO;
```

带条件的查询结果的分组（HAVING）

例15. 查选修10门课以上的学生学号。

```
SELECT SNO
FROM SC
GROUP BY SNO
HAVING COUNT(*)>=10;
```

注意：HAVING 决不等于WHERE，只能在GROUP BY 后出现！

3.4.1 简单查询 GROUP BY 子句

查询结果的分组（GROUP BY）

例15. 查平均成绩90分以上的学生学号和平均成绩。

```
SELECT SNO, AVG(Grade)
FROM SC
WHERE AVG(Grade) >= 90
GROUP BY SNO;
```

或直接写为:

```
SELECT SNO, AVG(Grade)
FROM SC
WHERE AVG(Grade) >= 90
```

注意: 上面的写法都是错误的, 因为WHERE中不能直接使用聚集函数作为条件表达式。应该写为:

```
SELECT SNO, AVG(Grade)
FROM SC
GROUP BY SNO
HAVING AVG(Grade) >= 90;
```

3.4.1 简单查询 LIMIT子句

LIMIT子句用于限制SELECT语句查询结果的行数（元组数）：

LIMIT <行数1> [OFFSET <行数2>];

语义是：取<行数1>，忽略前<行数2>行，作为查询结果。

OFFSET 可以省略，表示不忽略任何行。

在实际应用中，LIMIT子句经常和ORDER BY 并列的子句。

例16. 查平均成绩排名前十的同学的学号和平均成绩。

```
SELECT SNO, AVG(Grade)
FROM SC
GROUP BY SNO;
ORDER BY AVG(Grade) DESC
LIMIT 10;
```

例17. 查平均成绩排名3~7名的同学的学号和平均成绩。

```
SELECT SNO, AVG(Grade)
FROM SC
GROUP BY SNO;
ORDER BY AVG(Grade) DESC
LIMIT 5 OFFSET 2;
```